

Maximem Synap, Explained

What it is, the problem it actually solves, and a hands-on project that proves it — not just an SDK tour.

1. The problem, in plain terms

Every AI agent has to fit everything it's "thinking about" into a limited context window: the chat history so far, its instructions, any tools it's using, and whatever documents or facts it needs. The naive approach is to just keep appending — every new message gets sent along with the entire conversation so far. Two things break as that history grows:

- **It gets expensive fast.** If every turn re-sends the whole history, cost grows roughly quadratically with conversation length — turn 50 re-sends 49 turns' worth of tokens, every single time.
- **It doesn't survive across sessions.** Close the chat and start a new one, and the agent is a stranger again — even about things you told it five minutes ago in a different conversation.

The common fix — summarize old messages to keep things short — trades one problem for another: cost drops, but summaries quietly drop details, and eventually the agent gets something wrong because the fact it needed got compressed away. Researchers call this the **accuracy cliff**.

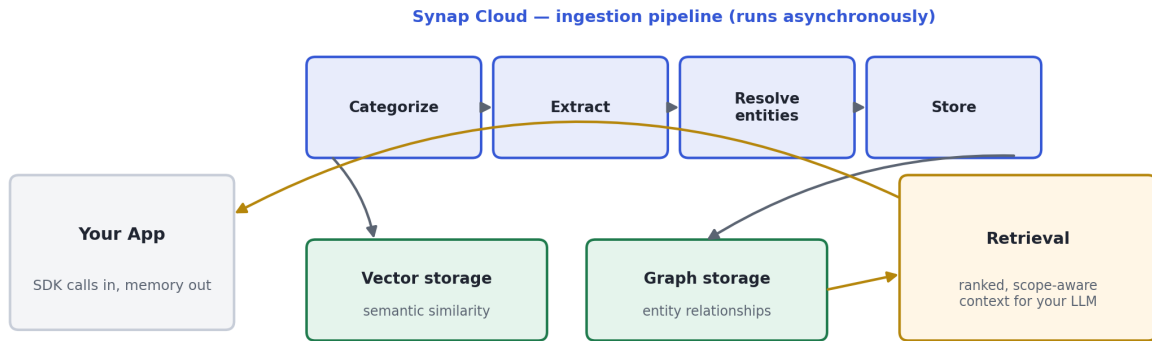
The analogy

Imagine a support agent who has no notes and no memory between calls. Every customer has to re-explain their whole setup from scratch, every single time — even if they called yesterday, even if a coworker at the same company already explained it. That's a chatbot with no memory layer.

2. What Maximem Synap actually does about it

Synap is a managed memory layer that sits between your application and your LLM. Instead of your app resending raw chat history, it hands Synap the conversation; Synap extracts the durable, reusable pieces (facts, preferences, events), stores them, and hands back only the small, relevant slice your agent needs for the current question.

Architecture at a glance



`memories.create(document, user_id, customer_id) → ingest` | `context.fetch(...) → retrieve`

Your app never talks to storage directly. It sends raw text in, and gets ranked, scope-aware context back.

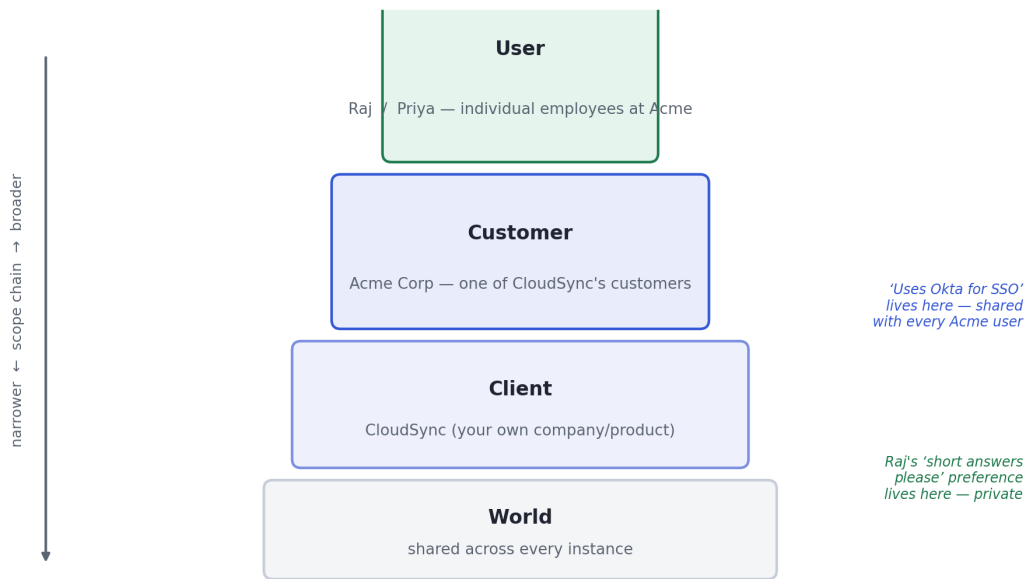
The six-step pipeline every message goes through

Step	What happens
1. Ingestion	Your app submits raw text (a chat turn, a document, a note), tagged with a scope.
2. Categorization	Synap classifies the content and decides which extraction rules apply.
3. Extraction	Pulls out structured facts, preferences, episodes, emotions, and dated events.
4. Entity resolution	"John", "John Smith", and "my manager" get merged into one entity in a knowledge graph.
5. Storage	Saved into both vector storage (semantic search) and graph storage (relationships).
6. Retrieval	When your agent needs context, Synap ranks and returns only what's relevant — respecting scope.

This whole pipeline runs **asynchronously** — your code gets an ingestion ID back immediately and can move on, rather than blocking on extraction.

3. The differentiator: memory that knows its boundaries

Plenty of tools can "remember" a single user. Synap's more interesting property is a **scope hierarchy** — User, Customer, Client, World — so a fact can be shared with exactly the right group of people, no more and no less.



Mapped onto a real scenario: a company-wide fact and a personal preference, kept in separate boxes on purpose.

This matters in practice: without it, you're stuck choosing between two bad options — give every user their own totally isolated memory (so coworkers repeat the same onboarding facts endlessly), or share one big memory pool (so one person's private preference leaks onto everyone else's replies).

The five identifiers you'll actually type

Identifier	Lives in	What it means
Client	Dashboard	Your own company's account. One per organization.
Instance	Dashboard	One deployed environment for an agent (prod, staging...). Own keys, own storage.
Customer (customer_id)	Every SDK call, B2B only	Your B2B tenant. Shared knowledge lives here. Auto-resolved on B2C, so you omit it.
User (user_id)	Every SDK call	Your end-user. Private memories live here.
Conversation (conversation_id)	Conversation-scoped calls	One chat thread. Must be a valid UUID.

B2C vs. B2B — the one setting that changes everything else

- **B2C** (personal app, one tier of users): identify by `user_id` only. `customer_id` is auto-resolved server-side.
- **B2B** (your customers are organizations with many users each): pass **both** `user_id` and `customer_id` on every call. This is what unlocks the shared-vs-private behavior above.

4. Hands-on project: TeamMind

A real problem, not an SDK tour: **CloudSync** is a B2B SaaS product. **Acme Corp** is one of its customers, with several employees who all contact support separately. Two things go wrong with a memory-less bot:

- Every new Acme employee has to re-explain company setup (SSO provider, admin contact, etc.) that a coworker already explained last week.
- If you fix that by sharing everything in one bucket, one employee's personal request ("give me terse answers, skip the small talk") leaks onto replies to their coworkers too.

TeamMind is a small, runnable Python project that proves Synap solves both at once, using its real SDK (not a simulation) against your own Synap instance.

What you'll actually see happen

Step	What it proves
1. Raj (employee #1) tells the bot Acme uses Okta for SSO, and separately mentions he personally prefers terse replies — in one message.	Synap must correctly split one message into a shared fact and a private preference.
2. Priya (employee #2, same company, never met Raj) asks how to set up SSO.	Priya's reply should already reflect the Okta fact Raj gave — with zero re-explaining.
3. The bot's context for Priya is printed before it answers.	Raj's "terse replies" preference must be absent from Priya's context — proving isolation, not just sharing.

Architecture of the project

File	Responsibility
config.py	Loads/validates environment variables once; fails fast with a clear message.
llm.py	Generates replies — real OpenAI call if configured, otherwise a template. Memory demo works either way.
memory_bot.py	The only file that imports the Synap SDK. Wraps ingestion, scoped retrieval, and retry logic.
run_demo.py	The scripted end-to-end proof described above. Run this first.
chat.py	Interactive CLI — have your own conversation as any employee at any company.

Setup

```
# 1. Create a Client -> Instance -> API key at synap.maximem.ai
#   Instance Settings -> User Relationship -> set to B2B
```

```
# 2. Install
pip install -r requirements.txt
```

```
# 3. Configure
export SYNAP_API_KEY=synap_your_key_here
# OPENAI_API_KEY is optional -- omit it to use template replies
```

```
# 4. Run the scripted proof
python run_demo.py
```

```
# 5. Or explore interactively
python chat.py --user raj --customer acme_corp
python chat.py --user priya --customer acme_corp
```

The core loop (from memory_bot.py)

```
async def ask(self, user_id, customer_id, conversation_id, user_message):
    # 1. Register the turn (needed for conversation-scoped retrieval later)
    await self._sdk.conversation.record_message(
        conversation_id=conversation_id, role="user",
        content=user_message, user_id=user_id, customer_id=customer_id)

    # 2. Fetch BOTH scopes explicitly -- this is the whole point
    company = await self.company_context(customer_id, [user_message])
    personal = await self.personal_context(user_id, customer_id, [user_message])

    # 3. Generate a reply using only what each scope actually returned
    reply = await llm.generate_reply(user_message,
        company.company_facts, personal.personal_preferences, ...)

    # 4. Persist the turn so new facts/preferences get extracted going forward
    await self._sdk.memories.create(document=f"User: {user_message}\nAssistant: {reply}",
        user_id=user_id, customer_id=customer_id, ...)
```

The full, runnable project (all five files, plus a README) is provided alongside this PDF.

5. Mentor notes — things worth knowing before you rely on this

A real discrepancy I found, not a hypothetical one

I didn't just read the docs — I downloaded the actual maximem-synap package (v0.4.0) from PyPI and read its source. The Quickstart's B2C example calls `sdk.conversation.record_message(...)` **without** `customer_id`. The installed SDK's actual method signature requires it (no default value) — that call would raise a `TypeError` as written. It doesn't affect TeamMind, since this project is B2B and always passes `customer_id` anyway — but if you later build a pure-B2C flow, verify this against whatever SDK version you actually install, don't assume the doc snippet runs as-is.

Other gaps worth tracking

- **Ingestion is asynchronous.** `memories.create()` returns immediately; the extracted facts aren't queryable until the pipeline finishes. The project calls `wait_for_completion()` for the demo's sake — a real production app would not block a user-facing request on this, and needs a plan for "the fact isn't ready yet."
- **Client-side merge, not atomic.** `ask()` does two separate fetches (company + personal) and merges them in your code. That's deliberate here, to make the isolation visible — but it's two network round-trips instead of one. Check whether a unified fetch interface exists in current docs before scaling this past a demo.
- **Error handling is a first cut, not exhaustive.** The wrapper retries once on `SynapTransientError` and fails fast on `SynapPermanentError` — real production code should add backoff with jitter, a retry budget, and almost certainly a circuit breaker if Synap becomes unavailable, since your agent's context quality now depends on a third-party service being up.
- **Privacy and data residence.** You're sending real conversation content to a third-party memory service. Before using this on anything beyond a demo, check Synap's data retention, region options, and whether cloud embeddings (if you enable them) send data to a further third party (OpenAI/Gemini/etc.).
- **The benchmark numbers you may see quoted elsewhere are self-reported.** Treat vendor-published accuracy/efficiency numbers as a starting hypothesis, not a guarantee for your own workload — test against your own data before trusting them for a production decision.

Where to go from here

- Run `run_demo.py` first — it's the fastest way to see the shared/private split actually happen, not just described.
- Then use `chat.py` to run your own scenario: a different fake company, different facts, and confirm the isolation holds up under conditions you chose, not just mine.
- Read the "Setup & Integration" and "Production Checklist" pages in the real docs before wiring this into anything beyond a local experiment.